LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 3&4 – FUNGSI & PROSEDUR

NAMA : Bambang Darmawan Saputra

NIM : 109082530003

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pemrograman Go (Golang), untuk membangun sebuah program yang terstruktur, bersih, dan mudah dikelola, digunakan beberapa konsep fundamental, di antaranya adalah subprogram (Fungsi dan Prosedur) serta metode pelewatan parameter.

1. Fungsi (Function) dan Prosedur (Procedure)

- Fungsi (Function): Merupakan blok kode yang dirancang untuk melakukan tugas tertentu dan mengembalikan sebuah nilai (*return value*) kepada pemanggilnya. Fungsi sangat berguna untuk membuat kode menjadi *reusable* (dapat digunakan berkali-kali) sehingga menghindari duplikasi kode. Dalam Golang, fungsi dideklarasikan dengan kata kunci func.
- Prosedur (Procedure): Konsep ini mirip dengan fungsi, namun pembedanya adalah prosedur tidak mengembalikan nilai (*no return value*). Dalam Golang, prosedur tetap dibuat menggunakan kata kunci func, tetapi tanpa mendefinisikan tipe data kembalian. Biasanya, prosedur digunakan untuk melakukan aksi langsung, seperti mencetak output ke layar atau memanipulasi variabel global/pointer.

2. Pelewatan Parameter (Parameter Passing)

- Pass by Value: Metode default dalam Golang. Saat variabel dikirim ke dalam fungsi, Golang hanya membuat salinan (copy) dari nilai variabel tersebut. Perubahan nilai yang terjadi di dalam fungsi tidak akan memengaruhi variabel asli di luar fungsi.
- Pass by Reference (Pointer): Metode ini mengirimkan alamat memori dari variabel asli, bukan salinan nilainya, menggunakan *pointer* (ditandai dengan operator & saat pengiriman dan * pada parameter). Karena fungsi mengakses alamat memori yang sama, setiap perubahan yang dilakukan di dalam fungsi akan secara permanen mengubah variabel asli di luar fungsi.

B. Guided

1. [perseegipanjang]

```go
package main

import "fmt"

func hitungLuasPersegiPanjang(panjang int, lebar int) int {
    luas := panjang * lebar
    return luas
}

func main() {
```

```go
    p := 10
    l := 5

    hasil := hitungLuasPersegiPanjang(p, l)

    fmt.Printf("Luas persegi panjang dengan panjang %d dan lebar %d
adalah: %d\n", p, l, hasil)
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run '.\Week 3- Modul 3&4\guided\guided1.go'
Luas persegi panjang dengan panjang 10 dan lebar 5 adalah: 50
```

Penjelasan :

Program Golang di atas berfungsi untuk menghitung luas persegi panjang dengan memanfaatkan sebuah fungsi (*function*) yang dapat digunakan kembali (*reusable*).

2. [cekbilangan]

```go
package main

import "fmt"

func cekGenap(angka int) bool {

    if angka%2 == 0 {
        return true
    }
    return false
}

func main() {
    angkaTes := 9

    hasilGenap := cekGenap(angkaTes)

    fmt.Printf("Apakah %d itu bilangan genap? Jawabannya: %t\n",
angkaTes, hasilGenap)
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul
3&4\Guided\Guided2.go"
Apakah 9 itu bilangan genap? Jawabannya: false
```

Penjelasan :

Program Golang ini berfungsi untuk mengecek apakah suatu bilangan adalah bilangan genap atau bukan, dan mengembalikan hasilnya dalam bentuk *boolean* (true atau false).

3. [perbedaan

```go
package main

import "fmt"

func tambahValue(x int) {
    x = x + 10
    fmt.Println("Nilai x didalam prosdur tambahValue (pass by value)
: ", x)
}

func tambahReference(x *int) {
    *x = *x + 10
    fmt.Println("Nilai x didalam prosdur tambahReference (pass by
refeence) : ", *x)
}

func main() {
    var y int = 5

    fmt.Println("nilai y awal : ", y)
    tambahValue(y)
    fmt.Println("nilai y setelah pass by value : ", y)

    tambahReference(&y)
    fmt.Println("nilai y setelah pass by reference : ", y)
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul
3&4\guided\guided3.go"
nilai y awal :  5
Nilai x didalam prosdur tambahValue (pass by value) :  15
nilai y setelah pass by value :  5
Nilai x didalam prosdur tambahReference (pass by refeence) :  15
nilai y setelah pass by reference :  15
```

Penjelasan :

Program Golang ini berfungsi untuk menunjukkan perbedaan penting antara mengirim data ke dalam fungsi menggunakan metode Pass by Value dan Pass by Reference (menggunakan *pointer*).

4. [belanja]

```go
package main

import "fmt"

func hitungTotalBelanja(harga int, jumlah int){
    var total int
    total = harga * jumlah
    fmt.Println("total harga : ", total)

    hitungDiskon(total)
}

func hitungDiskon(total int){
    var diskon, hargaAkhir int
    diskon = total * 10/100 //contoh diskon 10%
    hargaAkhir = total - diskon

    fmt.Println("Diskon 10% : ", diskon)
    fmt.Println("harga akhir (setelah diskon): ", hargaAkhir)
}

func main(){
    var harga, jumlah int

    fmt.Print("masukkan harga : ")
    fmt.Scan(&harga)
    fmt.Print("masukkan jumlah : ")
    fmt.Scan(&jumlah)

    hitungTotalBelanja(harga, jumlah)
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul 3&4\guided\guided4.go"
masukkan harga : 100
masukkan jumlah : 100
total harga :  10000
Diskon 10% :  1000
harga akhir (setelah diskon):  9000
```

Penjelasan :

Program Golang ini berfungsi untuk menghitung total biaya belanja dan otomatis menerapkan diskon sebesar 10%. Fokus utama dari kode ini adalah mendemonstrasikan bagaimana sebuah fungsi dapat memanggil fungsi lainnya untuk melanjutkan proses perhitungan.

C. Unguided

1. Soal Unguided 1

Minggu ini, mahasiswa Fakultas Informatika mendapatkan tugas dari mata kuliah matematika diskrit untuk mempelajari kombinasi dan permutasi. Jonas salah seorang mahasiswa, iseng untuk mengimplementasikannya ke dalam suatu program. Oleh karena itu buatlah program yang dapat menghitung kombinasi & permutasi.

Masukan terdiri dari empat buah bilangan asli a, b, c, d dengan syarat $a \geq c$ dan $b \geq d$

Keluaran terdiri dari empat baris. Baris pertama adalah hasil permutasi a terhadap c, baris kedua adalah hasil kombinasi a terhadap c, baris ketiga adalah hasil permutasi b terhadap d, dan baris keempat adalah hasil kombinasi b terhadap d.

Catatan Permutasi (P) dan kombinasi (C) dari n terhadap r ($n \geq r$) dapat dihitung dengan menggunakan persamaan berikut!

$$P(n,r) = \frac{n!}{(n-r)!}, \text{ sedangkan } C(n,r) = \frac{n!}{r!(n-r)!}$$

Selesaikan program tersebut dengan memanfaatkan subprogram yang diberikan dibawah ini :

```
Function factorial (n : integer) → integer
{I.S. terdefinisi bilangan bulat positif n
 F.S. mengembalikan nilai factorial dari n}

Function permutasi(n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n permutasi r}

Function kombinasi (n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n kombinasi r}
```

Hitunglah permutasi dan kombinasi apabila :

1) Nilai a = 5, Nilai b = 10, Nilai c = 3, Nilai d = 10
2) Nilai a = 15, Nilai b = 7, Nilai c = 8, Nilai d = 5

```go
package main

import "fmt"

func factorial(n int) int {
    if n == 0 || n == 1 {
```

```go
        return 1
    }
    result := 1
    for i := 2; i <= n; i++ {
        result *= i
    }
    return result
}

func permutasi(n, r int) int {
    return factorial(n) / factorial(n-r)
}

func kombinasi(n, r int) int {
    return factorial(n) / (factorial(r) * factorial(n-r))
}

func main() {
    var a, b, c, d int
    fmt.Print("masukkan nilai a: ")
    fmt.Scan(&a)
    fmt.Print("masukkan nilai b: ")
    fmt.Scan(&b)
    fmt.Print("masukkan nilai c: ")
    fmt.Scan(&c)
    fmt.Print("masukkan nilai d: ")
    fmt.Scan(&d)

    if a >= c && b >= d {
        fmt.Printf("hasil permutasi %d dan %d adalah: %d\n", a, c,
permutasi(a, c))
        fmt.Printf("hasil kombinasi %d dan %d adalah: %d\n", a, c,
kombinasi(a, c))
        fmt.Printf("hasil permutasi %d dan %d adalah: %d\n", b, d,
permutasi(b, d))
        fmt.Printf("hasil kombinasi %d dan %d adalah: %d\n", b, d,
kombinasi(b, d))
    } else {
        fmt.Println("Syarat a >= c dan b >= d tidak terpenuhi.")
    }
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul
3&4\unguided\3KombinasidanPermutasi.go"
masukkan nilai a: 5
masukkan nilai b: 10
masukkan nilai c: 3
masukkan nilai d: 10
hasil permutasi 5 dan 3 adalah: 60
hasil kombinasi 5 dan 3 adalah: 10
hasil permutasi 10 dan 10 adalah: 3628800
hasil kombinasi 10 dan 10 adalah: 1
```

Penjelasan :

Program ini meminta empat bilangan a, b, c, d dengan syarat a >= c dan b>= d. Program akan menghitung permutasi dan kombinasi menggunakan rumus matematika.

2. Soal Unguided 2

Diberikan tiga buah fungsi matematika yaitu :

- $f(x) = x^2$
- $g(x) = x - 2$
- $h(x) = x + 1$

Fungsi komposisi $(f\ o\ g\ o\ h)(x)$ artinya $f(g(h(x)))$. Tuliskan $f(x)$, $g(x)$, dan $h(x)$ dalam bentuk function.

Masukan terdiri dari sebuah bilangan bulat a, b, c

Keluaran terdiri dari 3 baris, baris pertama adalah $(f\ o\ g\ o\ h)(a)$, baris kedua adalah $(g\ o\ h\ o\ f)(b)$, dan baris ketiga adalah $(h\ o\ f\ o\ g)(c)$.

hitunglah apabila masukan :

1) Nilai a = 7, nilai b = 2, nilai c = 10
2) Nilai a = 15, nilai b = 8, nilai c = 3
3) Nilai a = 5, nilai b = 17, nilai c = 8

Jawaban :

```go
package main

import "fmt"

func f(x int) int {
    return x * x
}

func g(x int) int {
```

```go
    return x - 2
}

func h(x int) int {
    return x + 1
}

func main() {
    var x, y, z int
    fmt.Print("Masukkan nilai x : ")
    fmt.Scan(&x)
    fmt.Print("Masukkan nilai y : ")
    fmt.Scan(&y)
    fmt.Print("Masukkan nilai z : ")
    fmt.Scan(&z)

    fmt.Printf("F(G(H( %d ))) : %d\n", x, f(g(h(x))))
    fmt.Printf("G(H(F( %d ))) : %d\n", y, g(h(f(y))))
    fmt.Printf("H(F(G( %d ))) : %d\n", z, h(f(g(z))))
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul
3&4\unguided\3KomposisiFungsi.go"
Masukkan nilai x : 15
Masukkan nilai y : 8
Masukkan nilai z : 3
F(G(H( 15 ))) : 196
G(H(F( 8 ))) : 63
H(F(G( 3 ))) : 2
```

Penjelasan :

Program ini menerapkan tiga fungsi matematika sederhana , Program akan menerima tiga buah input lalu memprosesnya secara bersarang untuk menghasilkan komposisi fungsi.

3. Soal Unguided 3

Skiena dan Revilla dalam Programming Challenges mendefinisikan sebuah deret bilangan. Deret dimulai dengan sebuah bilangan bulat n. Jika bilangan n saat itu genap, maka suku berikutnya adalah ½n, tetapi jika ganjil maka suku berikutnya bernilai 3n+1. Rumus yang sama digunakan terus menerus untuk mencari suku berikutnya. Deret berakhir ketika suku terakhir bernilai 1. Sebagai contoh jika dimulai dengan n = 22, maka deret bilangan yang diperoleh adalah:

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

Untuk suku awal sampai dengan 1000000, diketahui deret selalu mencapai suku dengan nilai 1.

Buatlah program **skiena** yang akan mencetak setiap suku dari deret yang dijelaskan di atas untuk nilai suku awal yang diberikan. Pencetakan deret harus dibuat dalam prosedur cetakDeret yang mempunyai 1 parameter formal, yaitu nilai dari suku awal.

```
procedure cetakDeret(in n : integer)
```

Masukan berupa satu bilangan integer positif yang lebih kecil dari 1000000.

Keluaran terdiri dari satu baris saja. Setiap suku dari deret tersebut dicetak dalam baris yang dipisahkan oleh sebuah spasi.

Jawaban :

```go
package main

import "fmt"

func cetakDeret(n int) {
    for n != 1 {
        fmt.Printf("%d ", n)
        if n%2 == 0 {
            n = n / 2
        } else {
            n = (3 * n) + 1
        }
    }
    fmt.Printf("1\n")
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan : ")
    fmt.Scan(&n)
    cetakDeret(n)
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul 3&4\unguided\4DeretSkiena.go"
Masukkan bilangan : 22
22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

Penjelasan :

Deret ini memiliki aturan unik: jika bilangan genap maka akan dibagi 2 (atau menjadi 1/2n), dan jika ganjil akan dikalikan 3 ditambah 1 (3n+1). Deret ini akan terus berlanjut hingga mencapai angka 1. Kita akan menggunakan procedure cetak Deret untuk mencetak hasilnya dalam satu baris dengan spasi.

4. Soal Unguided 4

Buatlah sebuah program yang dapat menghitung luas & keliling persegi, persegi panjang, dan lingkaran. Setiap operasi perhitungan bangun datar dibuat prosedurnya sendiri-sendiri :

```
Procedure hitungPersegi(sisi : integer)
{I.S. terdefinisi bilangan bulat sisi
 F.S. menampilkan hasil perhitungan luas & keliling persegi}

Procedure hitungPersegiPanjang(panjang, lebar : integer)
{I.S. terdefinisi bilangan bulan Panjang & lebar
 F.S. menampilkan hasil perhitungan luas & keliling persegi
Panjang}

Procedure hitungLingkaran(jarijari : float64)
{I.S. terdifinisi bilangan decimal jarijari
 F.S. menampilkan hasil perhitungan luas & keliling lingkaran}
```

Buat menu tampilan awal program seperti berikut ini. (hint : gunakan percabangan switch-case)

```
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan :
```

Kemudian hitunglah luas & keliling :

1) Persegi dengan sisi 1057
2) Persegi Panjang dengan Panjang 106 dan lebar 88
3) Lingkaran dengan jari-jari 13,87

Jawaban :

```
package main
```

```go
import "fmt"

func hitungPersegi(sisi int) {
    luas := sisi * sisi
    keliling := 4 * sisi
    fmt.Printf("Luas persegi: %d\n", luas)
    fmt.Printf("Keliling persegi: %d\n", keliling)
}

func hitungPersegiPanjang(panjang, lebar int) {
    luas := panjang * lebar
    keliling := 2 * (panjang + lebar)
    fmt.Printf("Luas persegi panjang: %d\n", luas)
    fmt.Printf("Keliling persegi panjang: %d\n", keliling)
}

func hitungLingkaran(jarijari float64) {
    const pi = 3.14
    luas := pi * jarijari * jarijari
    keliling := 2 * pi * jarijari
    fmt.Printf("Luas lingkaran: %f\n", luas)
    fmt.Printf("Keliling lingkaran: %f\n", keliling)
}

func main() {
    var pilihan int
    fmt.Println("PROGRAM BANGUN DATAR")
    fmt.Println("1. Hitung luas & keliling persegi")
    fmt.Println("2. Hitung luas & keliling persegi panjang")
    fmt.Println("3. Hitung luas & keliling lingkaran")
    fmt.Print("Pilihan: ")
    fmt.Scan(&pilihan)

    switch pilihan {
    case 1:
        var sisi int
        fmt.Print("Masukkan sisi: ")
        fmt.Scan(&sisi)
        hitungPersegi(sisi)
    case 2:
        var p, l int
        fmt.Print("Masukkan panjang: ")
        fmt.Scan(&p)
        fmt.Print("Masukkan lebar: ")
```

```go
        fmt.Scan(&l)
        hitungPersegiPanjang(p, l)
    case 3:
        var r float64
        fmt.Print("Masukkan jari-jari: ")
        fmt.Scan(&r)
        hitungLingkaran(r)
    default:
        fmt.Println("Pilihan tidak valid")
    }
}
```

Output :

```
PS D:\github\109082530003_Bambang-Darmawan-Saputra-1> go run "d:\github\109082530003_Bambang-Darmawan-Saputra-1\Week 3- Modul 3&4\unguided\4Menghitung Luas danKelilingBangunDatar.go"
PROGRAM BANGUN DATAR
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan: 1
Masukkan sisi: 1057
Luas persegi: 1117249
Keliling persegi: 4228
```

Penjelasan :

Pada program ini, kita menggunakan konsep *procedure* (fungsi tanpa nilai kembalian) untuk langsung mencetak hasil dari operasi perhitungan persegi, persegi panjang, dan lingkaran. Program ini menggunakan menu interaktif dengan perulangan switch-case. Berdasarkan *test-case* pada contoh output (jari-jari 13.87 menghasilkan luas 604.063466), perhitungan menggunakan nilai Pi sebesar 3.14.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan Fungsi (*Function*) dan Prosedur (*Procedure*) sangat krusial dalam menciptakan program Golang yang modular, efisien, dan *reusable*, sehingga memudahkan proses pemeliharaan serta pencarian kesalahan (*debugging*) pada kode. Pemahaman mengenai perbedaan subprogram yang mengembalikan nilai (fungsi) dan yang langsung melakukan aksi (prosedur) sangat membantu dalam menyusun alur logika yang terstruktur. Selain itu, penguasaan konsep *Pass by Value* dan *Pass by Reference* menggunakan *pointer* terbukti penting untuk manajemen memori yang efisien serta memanipulasi nilai variabel asli secara permanen. Pada akhirnya, konsep modularitas ini dapat dikombinasikan secara bebas dengan berbagai struktur kontrol program seperti percabangan dan perulangan untuk menyelesaikan beragam masalah komputasi, mulai dari

perhitungan matematika dasar hingga pembuatan program interaktif berbasis menu.